Assignment 2: Data Transformation[1]

Mohamed Abdelmaksoud & Fedor Turchenko
Prof. Dr. Ziawasch Abedjan

[1]Data Integration: Algorithms and Systems
Technische Universität Berlin
Data Integration and Data Preparation (D2IP Lab)

Abstract. *The goal of this assignment is to transform data values from one format to another, which is called data transformation.*

Submission Guidelines

Submission Deadline: 16.11.2026 **Presentation:** 18.11.2026

How to submit:

- Submit a compressed file that contains (1) the PDF report and (2) your source code files via **ISIS**.
- One submission per each group is enough.
- You are free to use whatever tools/programming languages you intend to.
- Ensure that both group members' names are clearly listed at the beginning of your final PDF report.

1. Setup

1.1. Datasets

We have a relational dataset (Table 1) with the following schema:

$$Countries(Name, City, Code).$$

Here, *City* is the capital city of the country and *Code* is its three-letter ISO 3166-1 alpha-3 country code (e.g., DEU for Germany). In the attached file, the headers of these two columns are hidden (*Unknown_1*, *Unknown_2*). **Do not use the column headers in your solution.**

We have another dataset (Table 2) that lists the countries that placed **second** in the **Summer Olympic** medals table for each specified year, following the official Olympic ranking (gold, silver, bronze), with the following schema:

$$Olympics(Year, Country).$$

The datasets have been attached to this document.

Name	City	Code
Afghanistan	Kabul	AFG
Albania	Tirane	ALB
Algeria	Algiers	DZA
American Samoa	Pago Pago	ASM
Andorra	Andorra la Vella	AND
Angola	Luanda	AGO
Antigua and Barbuda		
Argentina		
⋮	⋮	⋮

Table 1. Excerpt of the *Countries* dataset

Year	Country
2012	China
1984	
2008	
1996	Russia
2000	
2024	
2016	
1980	German Democratic Republic

Table 2. Countries placed second in the Summer Olympic medals table by year (official Olympic ranking)

1.2. Ground Truth

To evaluate all methods in Section 2, build a **ground truth**: randomly select 30 countries with missing values in Table 1 and look up their capital and code manually in authoritative sources (e.g., the ISO 3166 list for the codes). A filled value is correct if it matches the ground truth; if no valid value exists (e.g., the country no longer exists), only an empty value is correct. *Accuracy* is the share of correct values among the 60 ground-truth cells.

2. Tasks

2.1. Data Transformation via a RESTful API

We want to fill the missing values inside the data columns *City* and *Code* in Table 1 with the help of a RESTful API[1].

1. Provide an algorithm. Specify the input, output, and time complexity.
2. Implement the algorithm and report the challenges.
3. Evaluate your results against the ground truth (Section 1.2).

You are only allowed to use the cell values provided in the table, **not the headers or the overall context**. There may be cases where you cannot fill in the missing values using the API. Identify these cases and explain the reason for each one.

2.2. Data Transformation via a Knowledge Base

We want to fill the missing values inside the data columns *City* and *Code* in Table 1 with the help of a knowledge base. You can either use a SPARQL endpoint, such as DBpedia's [2], or download a DBpedia dump, parse and query it locally.

1. Provide an algorithm. Specify the input, output, and time complexity.
2. Implement the algorithm and report the challenges.
3. Evaluate your results against the ground truth (Section 1.2).

You are only allowed to use the cell values provided in the table, **not the headers or the overall context**. There may be cases where you cannot fill in the missing values using the Knowledge Base. Identify these cases and explain the reason for each one.

2.3. Data Transformation using Large Language Models

In this task, you test a large language model (LLM) in three ways: (a) filling the missing values directly from examples, (b) writing a script that fills them, and (c) discovering an indirect transformation that combines both tables.

(a) Learning from Examples. The model fills *City* and *Code* in Table 1 from a varying number of examples.

1. Give the model the first $k \in \{0, 1, 3, 6\}$ complete rows of Table 1 as examples ($k = 0$: no complete row, only incomplete rows such as *Argentina,,*). Ask it to fill *City* and *Code* for all remaining countries and to explain in its own words what the missing values represent (e.g., "the second value is the capital of the country").
2. For each k, report the model's explanation, whether it is correct, and the accuracy on the ground truth (Section 1.2).

[1] For example, `https://restcountries.com`. Use the current API version (v5); older versions such as v3.1 are deprecated.
[2] `https://dbpedia.org/sparql`

3. List the wrong or invented values.

(b) Code Generation. The model writes a script instead of filling the values itself.

1. Give the model the six complete rows and ask it for a **script** that completes the table in the file *countries.csv*. Report which approach the script uses (e.g., an API, a knowledge base, a combination of both, or values hard-coded by the model).
2. Run the script unchanged. If it fails, fix it with minimal changes and list each error and the change you made.
3. Report the accuracy of the script's output on the ground truth (Section 1.2).

(c) Indirect Transformation. For each year of Table 2 with a missing country, the model finds the *capital* of the country that placed second (e.g., 2012 $\rightarrow$ China $\rightarrow$ Beijing).

1. *Two steps:* give the model Table 2 as it is, let it fill in the five missing countries, and then ask it for the capitals of these countries.
2. *Direct:* give the model a two-column table of years and capitals, **without the countries**: the three examples *2012, Beijing*; *1996, Moscow*; *1980, East Berlin*, and the five remaining years with an empty capital. Ask it to fill in the five capitals and to explain in its own words how the year and the capital are related.
3. Compare the accuracy of both ways (look up the correct answers in the official medal tables) and discuss whether the model's explanation in the direct way is correct.

Rules for (a)–(c).

- *Prompt content:* the rows as comma-separated values (e.g., *Afghanistan, Kabul, AFG*) and a neutral instruction, such as "Complete the missing values" or "Write a script that completes the missing values". **No headers, captions, or hints about the columns.**
- *Model:* any LLM without web search or tools. Report its name, version, date of use, and temperature (if available).
- *Documentation:* append all prompts and full outputs to your report.

2.4. Comparison

Summarize the accuracies of Sections 2.1 and 2.2 and of Section 2.3 (a) (for each k) and (b) in one table, and discuss the strengths and weaknesses of each method.